LAB-1203: Build an Embedded AI Agent in IBM Planning Analytics using watsonx Orchestrate

Hands-on lab guide

Aisha Mohammed Farooq Darga
AI Engineer, Service Engineering - US
Select Territory
aisdarg1@in.ibm.com



Table of Contents

What you will build

- Parts
- Conventions in this guide

Part 0: Prerequisites and security rules

- You need
- Security rules for this lab

Part 1: Set up the ADK and log in to wxO

- 1.1 Create the project folder and Python environment
- 1.2 Get your wxO service URL and API key
- 1.3 Add and activate the environment

Part 2: Connect the PA MCP server to wxO

- 2.1 Find the PA MCP server URL and API key
- 2.2 Create the connection
- 2.3 Register the toolkit with eight scoped tools
- 2.4 Look at the data (optional but useful)

Part 3: Build the agent (data and analysis only)

- 3.1 Create the file
- 3.2 Import the agent
- 3.3 Find it in the wxO UI

Part 4: Test the agent (data only)

- 4.1 Open the preview
- 4.2 Run these prompts, one message at a time, in the same conversation
- 4.3 Check the answers against the cube
- 4.4 Optional: see the tool calls

Part 5: Create the COS connection in wxO

- 5.1 Get the five values from your instructor
- 5.2 Create the wxO connection

Part 6: Build the report tool and add it to the agent

- 6.1 Create tools/requirements.txt
- 6.2 Create tools/publish_report_pdf.py
- 6.3 (Optional) Test the PDF code on your computer
- 6.4 Import the tool
- 6.5 Add the tool to the agent: three edits to the YAML

Part 7: Test the full report flow

- 7.1 Run the seven prompts in order, in one conversation
- 7.2 Check the PDF
- 7.3 Confirm the file is in COS

Part 8: Embed the agent in the PA workspace

- 8.1 Deploy the agent to the live environment
- 8.2 Generate an RSA key pair
- 8.3 Register the public key in wxO
- 8.4 Get the embed values
- 8.5 Create the configuration file
- 8.6 Enter the configuration in PA

8.7 Test inside PA
Things to know about the embedded agent

Part 9: Troubleshooting

Part 10: Clean-up

Appendix A: File layout at the end of the lab

Appendix B: Resources created

Appendix C: Command quick reference

IBM TechXchange 2026 / © 2026 IBM Corporation

Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed "as is" without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts.
In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply."

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled,
isolated environments. Customer examples are presented as illustrations of how those customers have used IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer's responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory requirements that may affect the customer's business and any actions the customer may need to take to comply with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (Continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. **IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.**

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

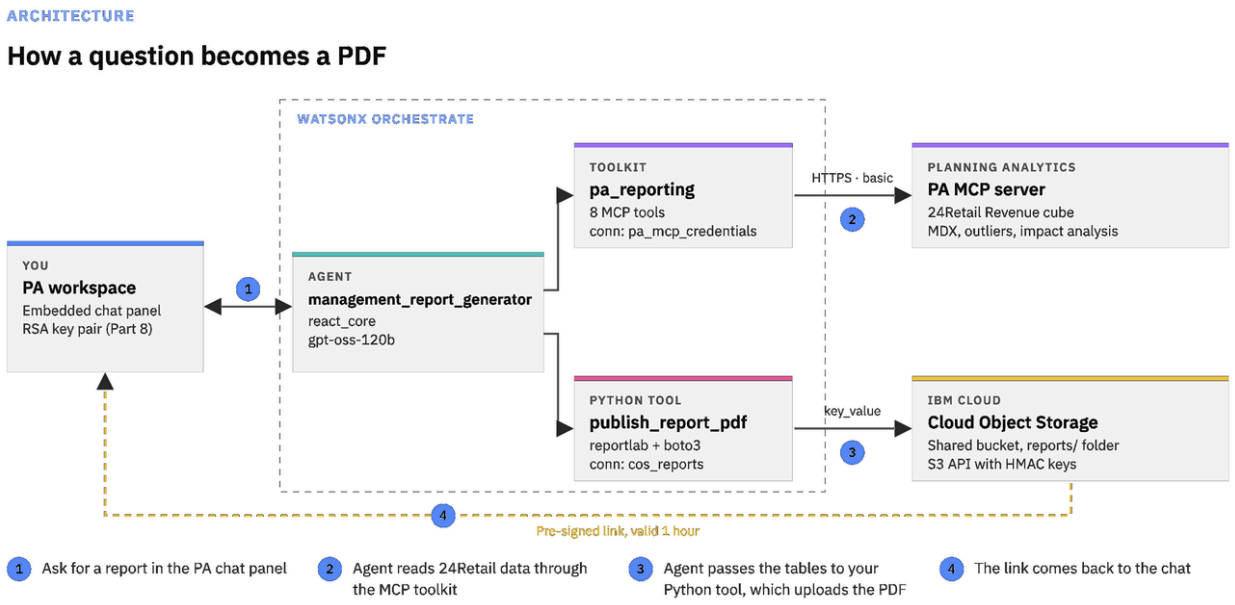
IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at “Copyright and trademark information” at: www.ibm.com/legal/copytrade.shtml.

LAB-1203: Build a Management Report Agent on watsonx Orchestrate and embed it in Planning Analytics

What you will build

A watsonx Orchestrate (wxO) agent that:

1. Reads live data from a Planning Analytics (PA) database through the PA MCP server.
2. Produces Budget vs Actual tables, outlier checks and variance-driver analysis.
3. Builds a PDF management report with a custom Python tool and stores it in IBM Cloud Object Storage (COS).
4. Returns a time-limited download link.
5. Runs inside the PA workspace (PAW) chat panel.



Parts

Part	What you do	Time
0	Prerequisites and security rules	5 min
1	Set up the ADK and log in to wxO	10 min
2	Connect the PA MCP server to wxO	15 min
3	Build the agent (data and analysis only)	15 min
4	Test the agent	15 min
5	Create the COS connection in wxO (shared bucket)	10 min
6	Build the report tool and add it to the agent	20 min
7	Test the full report flow	20 min
8	Embed the agent in the PA workspace	25 min
9	Troubleshooting	reference
10	Clean-up	5 min

Conventions in this guide

- Lines in grey code blocks are for your terminal (macOS zsh). Run them one block at a time.
- `<LIKE_THIS>` is a value you must replace.

- Menu names in the cloud consoles change between releases. Where the lab author could not confirm an exact label, the step says **(confirm label)**.

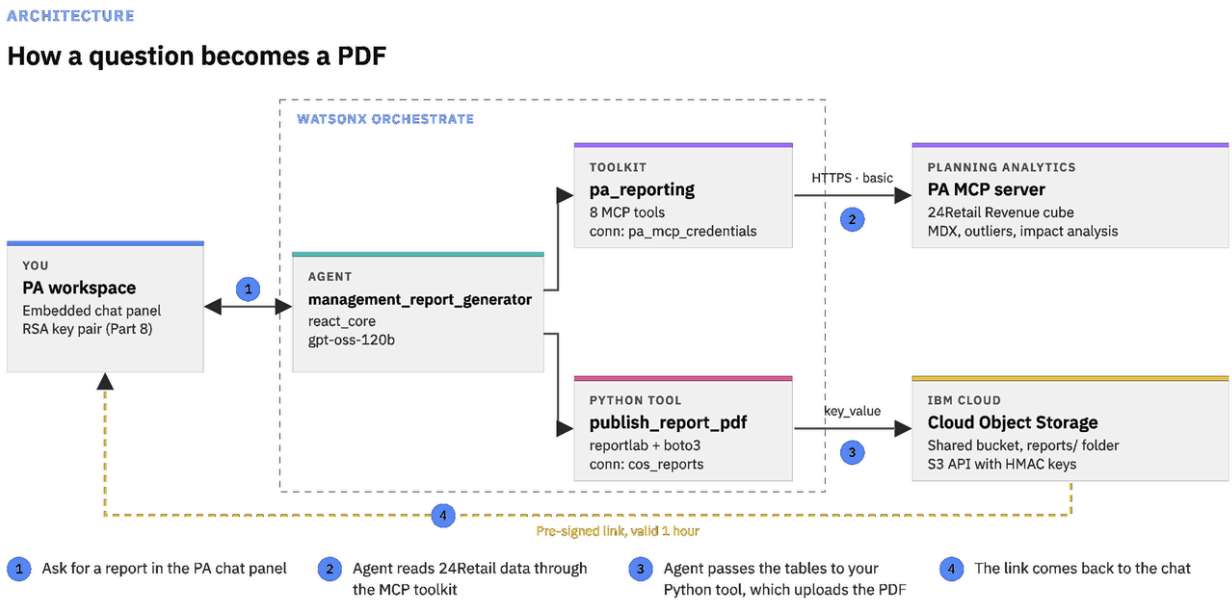
Part 0: Prerequisites and security rules

You need

Item	Notes
IBM Cloud account with a watsonx Orchestrate instance	The lab used region `ca-tor` (Toronto)
Shared COS bucket details	Five values from your instructor (Part 5.1). You do not need your own COS instance
Planning Analytics SaaS with the sample database **24Retail**	Plus an API key that can use the PA MCP server
Admin access to the PA Administration area	Needed in Part 8
A Mac or Linux terminal with Python 3.11 to 3.13	The lab used Python 3.13
`openssl`	Already installed on macOS

Security rules for this lab

1. **Never paste API keys, HMAC keys or private keys into a chat, ticket or screenshot.** Several keys in the original build were exposed this way and had to be rotated.
2. Enter secrets in the terminal with `read -s`. The text you type is not shown and is not saved in shell history.
3. Do not store secrets in files inside the project folder. If you must create one, delete it at the end (Part 10).
4. Use credentials made for this lab and delete them afterwards.



Part 1: Set up the ADK and log in to wxO

1.1 Create the project folder and Python environment

```
mkdir lab-1203 && cd lab-1203
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install ibm-watsonx-orchestrate
```

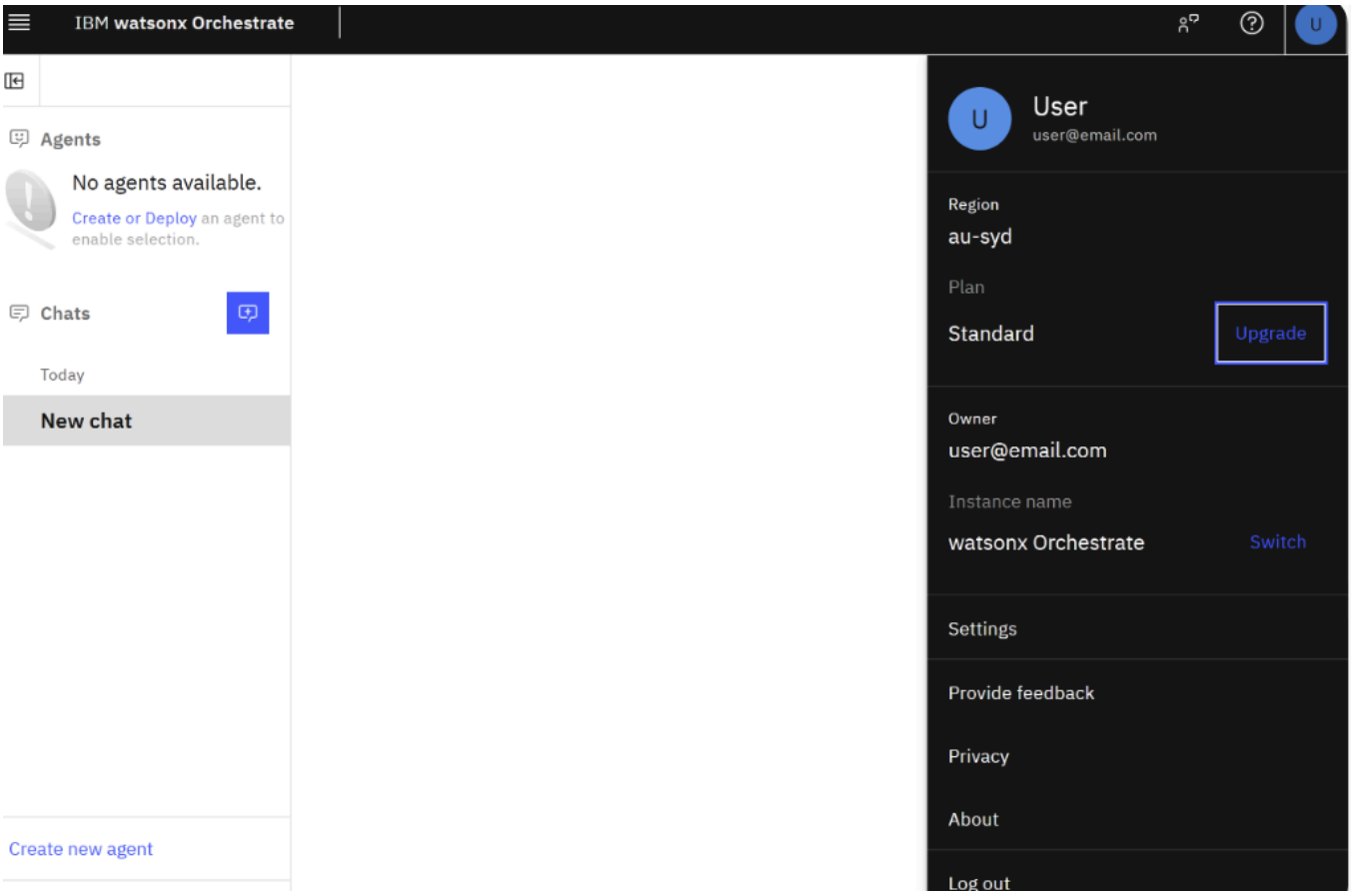
Check it works:

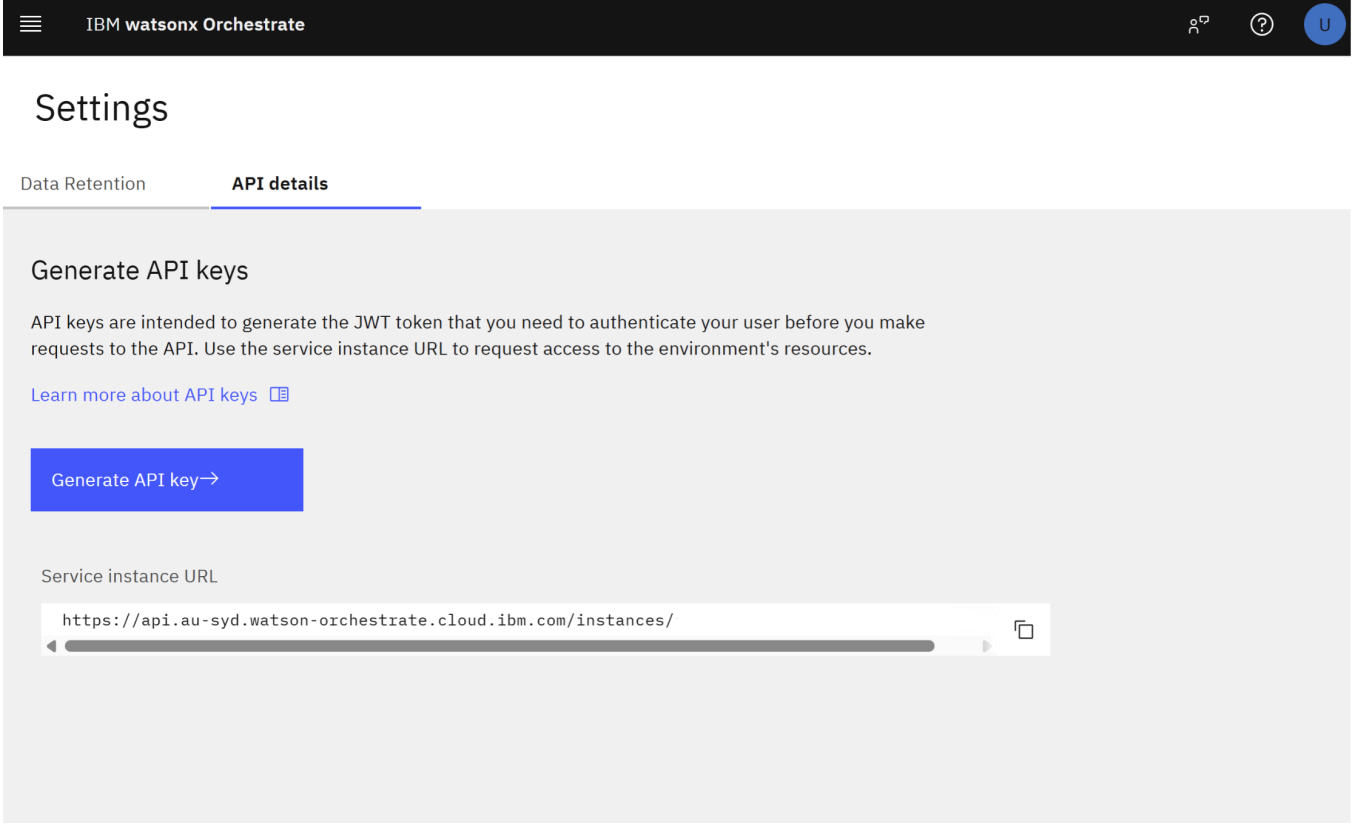
```
orchestrate --version
```

Every time you open a new terminal, run `cd lab-1203` and `source .venv/bin/activate` first.

1.2 Get your wxO service URL and API key

1. Sign in to IBM Cloud and open your **watsonx Orchestrate** instance.
2. Open the wxO UI. Click your profile icon, then **Settings**, then **API details**. **(confirm label)**
3. Copy the **Service instance URL**. It looks like `https://api.ca-tor.watson-orchestrate.cloud.ibm.com/instances/<INSTANCE_ID>`.
4. Click **Generate API key (confirm label)** and keep the key open. You will type it into the terminal in the next step.





1.3 Add and activate the environment

Add the environment. Use any name you like. The lab used `lab-1203`.

```
orchestrate env add -n lab-1203 -u "<SERVICE_INSTANCE_URL>" --type ibm_iam
```

Store the API key in a hidden variable. Run this line **on its own**, then paste the key and press Enter:

```
read -s WXO_KEY
```

Activate:

```
orchestrate env activate lab-1203 --api-key "$WXO_KEY"
```

Remove the variable:

```
unset WXO_KEY
```

Check:

```
orchestrate connections list
```

You should see a table (it may be empty).

> **Token expiry.** The login lasts a limited time. If any command says `The token found for environment 'lab-1203' is missing or expired`, repeat the `read -s`, `activate` and `unset` steps above.

Part 2: Connect the PA MCP server to wxO

A **toolkit** in wxO is a bundle of tools published by an MCP server. A **connection** holds the credentials. Remote MCP servers need a **Basic** connection with these values:

Field	Value
Username	`apiKey` (the literal word)
Password	your PA API key

> **Do not use a `key_value` connection for a remote MCP toolkit.** The first attempt in this lab used `key_value` and the server answered `401 Unauthorized`.

2.1 Find the PA MCP server URL and API key

The PA MCP server URL has this shape:

```
https://<PA_HOST>/api/<TENANT_ID>/v0/agentic-ai/ibm-pa-tools/mcp
```

Your instructor provides `<PA_HOST>`, `<TENANT_ID>` and the PA API key.

Instructor to confirm: the PA screen where students create or copy the API key, and the exact URL.

If you already have an MCP client file (for example `.bob/mcp.json`) with an `Authorization: Basic <base64>` header, the password is the part after the first colon once the base64 text is decoded. You can extract it into a hidden variable with this command (it prints nothing):

```
PA_KEY=$(python3 -c 'import json,base64;d=json.load(open(".bob/mcp.json"))["mcpServers"]["ibm-pa-tools"]["headers"]["Authorization"].split(" ",1)[1];print(base64.b64decode(d).decode().split(":",1)[1])')
```

Otherwise, type it in by hand:

```
read -s PA_KEY
```

2.2 Create the connection

Create the connection record:

```
orchestrate connections add -a pa_mcp_credentials
```

Configure it for both environments. The `--server-url` is the PA MCP URL:

```
for env in draft live; do orchestrate connections configure -a pa_mcp_credentials --env $env --type team --kind basic --server-url "<PA_MCP_URL>"; done
```

Set the credentials:

```
for env in draft live; do orchestrate connections set-credentials -a pa_mcp_credentials --env $env -u apiKey -p "$PA_KEY"; done
```

Clear the variable:

```
unset PA_KEY
```

Check:

```
orchestrate connections list
```

`pa_mcp_credentials` should show as configured for draft and live.

2.3 Register the toolkit with eight scoped tools

The PA server offers about 50 tools. A small agent works better with a short, focused list, so import only the tools it needs.

```
orchestrate toolkits add --kind mcp --name pa_reporting --description "Planning Analytics read-only reporting tools" --url "<PA_MCP_URL>" --transport streamable_http --tools "execute_mdx_and_get_view,get_cube_sample_members,lookup_potential_members,perform_outlier_detection,get_outlier_summary,perform_impact_analysis,get_impact_analysis_summary,generate_exploration_analysis_report" --app-id pa_mcp_credentials
```

Check:

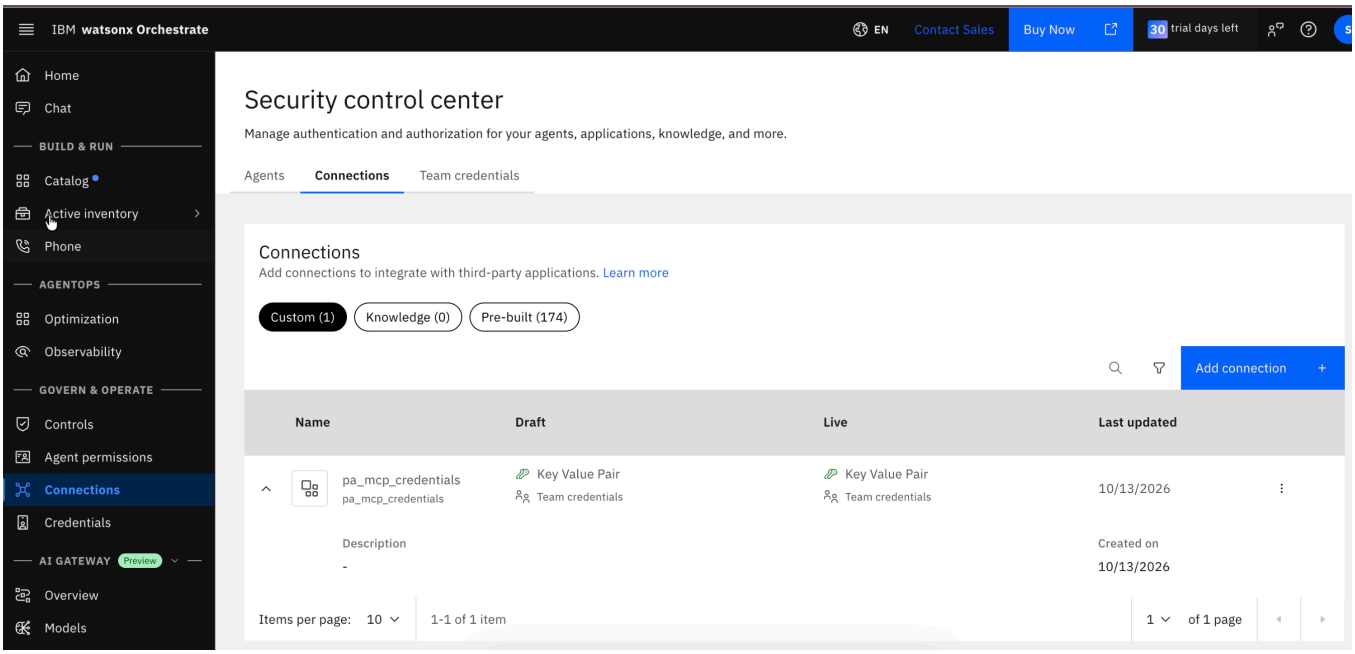
```
orchestrate toolkits list
orchestrate tools list | grep pa_reporting
```

Tool names have the form `pa_reporting:<tool>`.

> `generate_exploration_analysis_report` is imported here but the final agent does **not** use it. Part 6 replaces it with a custom Python tool.

2.4 Look at the data (optional but useful)

In wxO, open **Manage agents** or the **Tools** catalog **(confirm label)** and find `pa_reporting`. Open `execute_mdx_and_get_view` and read its description.



About the 24Retail Revenue cube

Dimension	Members used in this lab
organization	Total Company, East Region (100), Central Region (200), West Region (300), Canada (400)
Channel	Channel Total, Retail (10), Internet (20), Distribution (30)
product	Product Total, Phones (20000), PCs (30000), Tablets (40000)
Month	Year (full year), Q1 to Q4, Jan to Dec
Year	Y1 = 2022, Y2 = 2023, Y3 = 2024
Version	Actual, Version 1 (Budget), Forecast, Target, Prior Year Actual
Revenue (measures)	Gross Revenue, Cost of Sales, Gross Margin, Gross Margin %, Units Sold

The cube does not state its currency, so the agent must not print `$`.

Part 3: Build the agent (data and analysis only)

In this part the agent can read data, find outliers and explain variance. PDF output is added in Part 6.

3.1 Create the file

```
mkdir -p agents tools
```

Create `agents/management_report_generator.agent.yaml` with this exact content:

```
spec_version: v1
kind: native
name: management_report_generator
description: >
  Generates Budget vs Actual management reports for the 24Retail Planning
  Analytics database. Pulls revenue and margin data from the Revenue cube, flags
  anomalies and explains variance drivers.
llm: groq/openai/gpt-oss-120b
style: react_core
instructions: |
  You are the 24Retail Management Report Generator, an agent operating within watsonx
Orchestrate.
  You produce finance management reports from the Planning Analytics server "24Retail".

  Behavior and sources:
  - Always use the available tools to retrieve data before relying on your own knowledge.
  - Never invent numbers. If a tool does not return a value, say so.
  - If period, version or entity is missing, ask for all missing items in a single question.
    Defaults when the user does not say: entity Total Company, Channel Total, Product Total,
version Actual compared with Version 1 (Budget).
  - Label all narrative commentary "Draft - for review".

  Data rules for the Revenue cube on server 24Retail:
  - Dimensions: organization, Channel, product, Month, Year, Version, Revenue.
  - Year members: Y1 = 2022, Y2 = 2023, Y3 = 2024.
  - Version members include Actual, Version 1 (Budget), Forecast, Target.
  - Revenue measures include Units Sold, Gross Revenue, Cost of Sales, Gross Margin, Gross
Margin %.
  - Month member "Year" is the full-year total.
  - Use pa_reporting:lookup_potential_members or pa_reporting:get_cube_sample_members when
unsure of a member name.

  Tool usage rules:
  - Use pa_reporting:execute_mdx_and_get_view to read data. Keep result sets small (summary
level first, drill down only when asked).
  - Use pa_reporting:perform_outlier_detection and pa_reporting:get_outlier_summary on the view
state returned by execute_mdx_and_get_view to find unusual values. Always report outliers in an
Exceptions section.
  - Use pa_reporting:perform_impact_analysis and pa_reporting:get_impact_analysis_summary to
explain what drives a variance.
  - Before building any breakdown table (region, channel, product), verify that the rows add up
to the Total Company figure within 0.1 M. If they do not, re-query the mismatched member using
its cube member ID. Region member IDs are: East Region=100, Central Region=200, West
Region=300, Canada=400. Always use member IDs, not aliases, in MDX.
  - Call tools with all available information. If a tool returns an error, read the message and
retry with corrections. After 2 failed attempts, tell the user and suggest alternatives.

  Report structure (for full reports):
  1. Executive summary: headline Gross Revenue, Gross Margin %, variance versus Budget.
  2. Actual versus Budget, Forecast and Target table with variance and variance percent.
  3. Margin analysis: units sold, cost of sales, gross margin.
  4. Exceptions: outliers and data quality warnings.
```

```
  5. Draft commentary on the variance drivers.

  Formatting:
  - Always show monetary and unit amounts in millions (M) with one decimal, for every row in
every table without exception.
  - Do not assume a currency symbol. The cube does not state its currency, so omit "$" unless
the user supplies one.
  - Whenever two versions or years are compared, always include Variance and Variance %
columns.
  - Calculate Variance and Variance % from the full-precision values returned by the tool, then
round to one decimal only for display. Never subtract numbers that are already rounded.
  - When reporting outliers, list each flagged value once, make the count match the rows
listed, and keep every table row to the same number of columns.
  - Variance (M) = Version value minus Budget value. Variance % = (Variance / Budget) × 100.
Compute these from the full-precision numbers returned by the tools, then round to one decimal
for display. These two columns may be computed; all other columns must come directly from tool
results.
  - Never label a value as "derived". Retrieve all measures directly from tool results. If a
tool did not return a value, call the tool again or state the value is unavailable.
  - Keep chat replies concise. A full report may be longer.
tools:
  - pa_reporting:execute_mdx_and_get_view
  - pa_reporting:get_cube_sample_members
  - pa_reporting:lookup_potential_members
  - pa_reporting:perform_outlier_detection
  - pa_reporting:get_outlier_summary
  - pa_reporting:perform_impact_analysis
  - pa_reporting:get_impact_analysis_summary
collaborators: []
```

Why these choices

Setting	Reason
`llm: groq/openai/gpt-oss-120b`	The model used in the lab
`style: react_core`	This model forces `react_core`. The `planner` style does not apply to it
7 tools, no collaborators	A small tool list keeps the model accurate
Region IDs in the instructions	Without them the agent once looked up "Canada" by alias and returned the wrong number
Variance rule	Two earlier rules contradicted each other ("never compute figures" and "always show variance"). The model then showed `N/A`

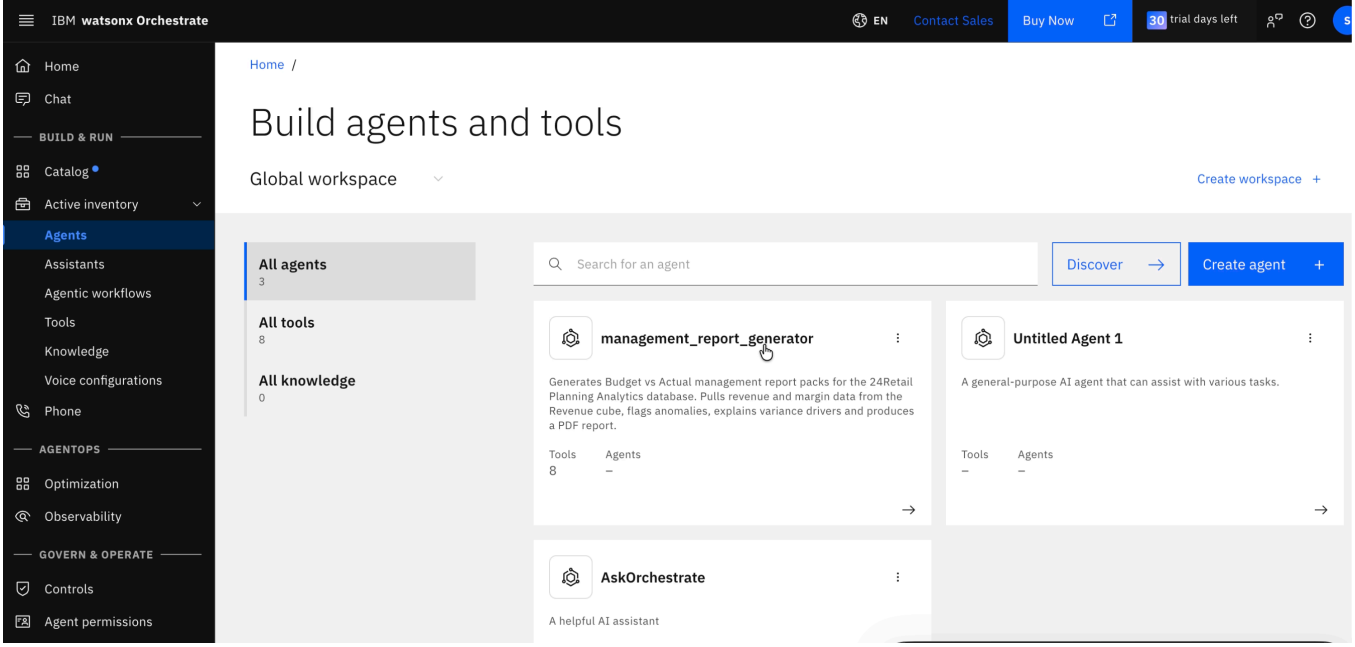
3.2 Import the agent

```
orchestrate agents import -f agents/management_report_generator.agent.yaml
```

Expected output ends with `Agent 'management_report_generator' created successfully`.

3.3 Find it in the wxO UI

1. Open the wxO UI.
2. Open the left menu and choose **Build** then **Agent Builder (confirm label)**.
3. Select **management_report_generator**.
4. On the agent page, check the **Toolset** section lists the seven `pa_reporting` tools.



Part 4: Test the agent (data only)

> Test in the wxO browser **preview** panel. The command `orchestrate chat ask` often times out after 90 to 180 seconds on the first call, so avoid it.

4.1 Open the preview

On the agent page, use the **Preview** panel on the right **(confirm label)**.

Screenshot: Preview panel with an empty chat

4.2 Run these prompts, one message at a time, in the same conversation

Prompt 1: executive summary

```
Show me Total Company full-year 2024 Gross Revenue, Gross Margin and Gross Margin % for Actual, Budget (Version 1) and Prior Year Actual. Amounts in millions with one decimal. Include Variance vs Budget (M) and Variance % vs Budget columns.
```

Prompt 2: regional breakdown

```
Break down 2024 full-year Actual and Budget Gross Revenue by region using member IDs 100 (East Region), 200 (Central Region), 300 (West Region), 400 (Canada). Millions, with Variance vs Budget and Variance %. Check that the four regions sum to Total Company 76.1 M.
```

Prompt 3: outliers

```
Run outlier detection on the 2024 Total Company full-year data across Actual, Budget, Prior Year Actual, Forecast and Target. Summarise what is flagged.
```

Prompt 4: variance drivers

```
Run impact analysis on Actual vs Budget for 2024 and summarise the top variance drivers.
```

4.3 Check the answers against the cube

These values were checked directly in the PA cube (Gross Revenue, full year 2024):

Member	Actual	Budget (Version 1)
Total Company	76,101,225	88,464,390
East Region (100)	20,100,102	25,228,094
Central Region (200)	13,998,373	16,068,012
West Region (300)	19,131,048	21,659,038
Canada (400)	22,871,703	25,509,245
Total Company, Prior Year Actual	73,606,160	

In millions with one decimal, the regions must add up to Total Company: 20.1 + 14.0 + 19.1 + 22.9 = 76.1 for Actual and 25.2 + 16.1 + 21.7 + 25.5 = 88.5 for Budget.

Known data quirks (not agent errors)

- Target 2023 is about 713.5 M and Forecast 2022 is about 217.9 M. These are far from the other years and are the outliers the detector should flag if you include those years.
- Variance may differ by 0.1 from a hand calculation on rounded numbers. The agent is told to compute from full-precision values.

Screenshot: Preview showing the Actual vs Budget table

4.4 Optional: see the tool calls

In the preview, expand the steps (shown as "Step 1, Step 2..." under the answer) to see which tools were called and with what MDX **(confirm label)**.

Part 5: Create the COS connection in wxO

In Part 6 you will build a custom Python tool that:

1. creates the PDF,
2. uploads it to Cloud Object Storage (COS),
3. returns a link.

All students use **one shared COS bucket** that the instructor created. You do not create a bucket or a credential. You only store the instructor's values in a wxO connection.

5.1 Get the five values from your instructor

Key	Value
`COS_ENDPOINT`	given by the instructor
`COS_REGION`	given by the instructor (for example `ca-tor`)
`COS_BUCKET`	given by the instructor
`COS_ACCESS_KEY_ID`	given by the instructor
`COS_SECRET_ACCESS_KEY`	given by the instructor

Treat the access key and secret like a password. Do not paste them into chat, tickets or screenshots.

Because the bucket is shared, every student's reports go to the same `reports/` folder. File names contain a timestamp and a random id, so they do not overwrite each other. The agent gives you a link to your own file, valid for 1 hour.

5.2 Create the wxO connection

The Python tool reads a `key_value` connection named `cos_reports` with the five entries from 5.1.

Create and configure it:

```
orchestrate connections add -a cos_reports
orchestrate connections configure -a cos_reports --env draft --type team --kind key_value
orchestrate connections configure -a cos_reports --env live --type team --kind key_value
```

Enter the two secrets without displaying them. Run each `read -s` line **on its own**:

```
read -s COS_AK
```

```
read -s COS_SK
```

Set the credentials:

```
for e in draft live; do orchestrate connections set-credentials -a cos_reports --env $e -e
"COS_ENDPOINT=<ENDPOINT>" -e "COS_REGION=<REGION>" -e "COS_BUCKET=<BUCKET>" -e
"COS_ACCESS_KEY_ID=$COS_AK" -e "COS_SECRET_ACCESS_KEY=$COS_SK"; done
```

```
unset COS_AK COS_SK
```

Check:

```
orchestrate connections list
```

`cos_reports` should show `key_value` with credentials set for draft and live.

Part 6: Build the report tool and add it to the agent

6.1 Create `tools/requirements.txt`

```
reportlab
boto3
```

6.2 Create `tools/publish_report_pdf.py`

```python
import io
import uuid
from datetime import datetime, timezone
from typing import List

from pydantic import BaseModel, Field
from ibm_watsonx_orchestrate.agent_builder.tools import tool
from ibm_watsonx_orchestrate.agent_builder.connections import ConnectionType
from ibm_watsonx_orchestrate.run import connections

COS_APP_ID = "cos_reports"
LINK_TTL_SECONDS = 3600

class ReportTable(BaseModel):
    heading: str = Field(description="Section heading shown above the table.")
    columns: List[str] = Field(description="Column headers.")
    rows: List[List[str]] = Field(
        description="Table rows. Every row must have the same number of cells as columns. All
values are strings, already formatted (for example '92.6')."
    )
```

```
    summary: str = Field(
        default="",
        description=(
            "Optional 2-4 sentence explanation shown above the table. "
            "State what the table shows, call out the biggest variance and explain why it
matters. "
            "Do not repeat numbers already in the table rows."
        ),
    )

_CHAR_MAP = {
    "\u2010": "-", "\u2011": "-", "\u2012": "-", "\u2212": "-",
    "\u00a0": " ", "\u202f": " ", "\u2009": " ", "\u200b": "",
}

def _safe(text: str) -> str:
    """Replace characters the built-in Helvetica font cannot draw (they render as black
boxes)."""
    for src, dst in _CHAR_MAP.items():
        text = text.replace(src, dst)
    return text.encode("cp1252", errors="replace").decode("cp1252")

def _inline(text: str) -> str:
    """Escape XML, then turn **bold** and *italic* markdown into reportlab tags."""
    import re
    from xml.sax.saxutils import escape

    out = escape(_safe(text))
    out = re.sub(r"\*\*(.+?)\*\*", r"<b>\1</b>", out)
    out = re.sub(r"(?<!\*)\*(?!\s)(.+?)(?<!\s)\*(?!\*)", r"<i>\1</i>", out)
    return out

def _build_pdf(title: str, subtitle: str, tables: List[ReportTable], commentary: str) ->
bytes:
    import re
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import getSampleStyleSheet
    from reportlab.lib.units import mm
    from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle

    buffer = io.BytesIO()
    doc = SimpleDocTemplate(
        buffer, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm,
        topMargin=18 * mm, bottomMargin=18 * mm, title=title,
    )
    styles = getSampleStyleSheet()
    story = [Paragraph(_inline(title), styles["Title"])]
    if subtitle:
        story.append(Paragraph(_inline(subtitle), styles["Normal"]))
    story.append(Spacer(1, 6 * mm))

    col_width = (A4[0] - 36 * mm) / 6

    for t in tables:
        story.append(Paragraph(_inline(t.heading), styles["Heading2"]))
        if t.summary:
            story.append(Paragraph(_inline(t.summary), styles["Normal"]))
            story.append(Spacer(1, 3 * mm))
        data = [[_safe(c) for c in t.columns]] + [[_safe(c) for c in r] for r in t.rows]
        n_cols = len(t.columns)
        col_widths = [col_width * 2] + [col_width] * (n_cols - 1) if n_cols > 1 else None
        table = Table(data, repeatRows=1, colWidths=col_widths)
        table.setStyle(TableStyle([
```

```
            ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#161616")),
            ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
            ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
            ("FONTSIZE", (0, 0), (-1, -1), 9),
            ("ALIGN", (1, 0), (-1, -1), "RIGHT"),
            ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
            ("GRID", (0, 0), (-1, -1), 0.25, colors.HexColor("#c6c6c6")),
            ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f4f4f4")]),
            ("TOPPADDING", (0, 0), (-1, -1), 4),
            ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
        ]))
        story.append(table)
        story.append(Spacer(1, 8 * mm))

    if commentary:
        story.append(Paragraph("Executive Commentary - Draft - for review",
styles["Heading2"]))
        for para in commentary.split("\n"):
            line = para.strip()
            bare = re.sub(r"[\s*_.\-:]+", " ", _safe(line)).strip().lower()
            if not line or bare in ("draft for review", "executive commentary draft for
review"):
                continue
            if line[:2] in ("- ", "* "):
                line = "\u2022 " + line[2:]
            story.append(Paragraph(_inline(line).replace("\u2022", "&bull;"),
styles["Normal"]))
            story.append(Spacer(1, 2 * mm))

    generated = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC")

    def footer(canvas, _doc):
        canvas.saveState()
        canvas.setFont("Helvetica", 8)
        canvas.drawString(18 * mm, 10 * mm, f"Generated {generated} - AI-assisted - Draft -
for review")
        canvas.drawRightString(A4[0] - 18 * mm, 10 * mm, f"Page {_doc.page}")
        canvas.restoreState()

    doc.build(story, onFirstPage=footer, onLaterPages=footer)
    return buffer.getvalue()

@tool(
    name="publish_report_pdf",
    description=(
        "Builds a PDF management report from already-retrieved table data and commentary,
uploads it to "
        "IBM Cloud Object Storage and returns a time-limited download link. Use only after the
data has been "
        "retrieved from Planning Analytics."
    ),
    expected_credentials=[{"app_id": COS_APP_ID, "type": ConnectionType.KEY_VALUE}],
)
def publish_report_pdf(
    title: str,
    tables: List[ReportTable],
    commentary: str = "",
    subtitle: str = "",
) -> dict:
    """Create a PDF report, store it in Cloud Object Storage and return a download link.

    Args:
        title (str): Report title, for example '24Retail FY2024 Management Report'.
        tables (List[ReportTable]): Tables to include. Each table has a heading, columns, rows
```

```
and an optional summary. The summary is a 2-4 sentence explanation shown above the table: state
what the table shows, call out the biggest variance and explain its significance. Do not repeat
numbers already in the rows.
        commentary (str): Full executive commentary for the end of the report. Write 1-2
paragraphs covering overall performance, key risks and outlook. Separate paragraphs with a
newline. Do not include the literal phrase 'Draft - for review' — the tool adds that label
automatically.
        subtitle (str): One line under the title, for example the entity, version and period.

    Returns:
        dict: download_url (valid for one hour), file_name and expires_in_minutes.
    """
    import boto3
    from botocore.config import Config

    # Normalise: the runtime may deliver each table as a plain dict instead of a ReportTable
object.
    normalised = []
    for t in tables:
        if isinstance(t, dict):
            normalised.append(ReportTable(**t))
        else:
            normalised.append(t)
    tables = normalised

    for t in tables:
        for r in t.rows:
            if len(r) != len(t.columns):
                raise ValueError(
                    f"Table '{t.heading}' has a row with {len(r)} cells but {len(t.columns)}
columns."
                )

    creds = connections.key_value(COS_APP_ID)
    client = boto3.client(
        "s3",
        endpoint_url=creds["COS_ENDPOINT"],
        aws_access_key_id=creds["COS_ACCESS_KEY_ID"],
        aws_secret_access_key=creds["COS_SECRET_ACCESS_KEY"],
        config=Config(signature_version="s3v4", s3={"addressing_style": "path"}),
        region_name=creds.get("COS_REGION", "us-south"),
    )
    bucket = creds["COS_BUCKET"]

    pdf_bytes = _build_pdf(title, subtitle, tables, commentary)
    stamp = datetime.now(timezone.utc).strftime("%Y%m%d-%H%M%S")
    key = f"reports/{stamp}-{uuid.uuid4().hex[:8]}.pdf"

    client.put_object(Bucket=bucket, Key=key, Body=pdf_bytes, ContentType="application/pdf")
    url = client.generate_presigned_url(
        "get_object",
        Params={
            "Bucket": bucket,
            "Key": key,
            "ResponseContentType": "application/pdf",
            "ResponseContentDisposition": f'inline; filename="{key.split("/")[-1]}"',
        },
        ExpiresIn=LINK_TTL_SECONDS,
    )
    return {"download_url": url, "file_name": key, "expires_in_minutes": LINK_TTL_SECONDS //
60}
```

How the tool works

Piece	What it does
`@tool(... expected_credentials=...)`	Declares that the tool needs the `cos_reports` key_value connection
`ReportTable`	The shape of each table the agent passes in: heading, columns, rows, summary
dict-to-`ReportTable` loop	The wxO runtime passes tables as plain dictionaries, which caused `AttributeError: 'dict' object has no attribute 'rows'` until this loop was added
`_safe`	Replaces characters Helvetica cannot draw. A non-breaking hyphen printed as a black box before this
`_inline`	Turns `**bold**` and `*italic*` markers into real formatting
`generate_presigned_url`	Makes a private link that expires after 3600 seconds

6.3 (Optional) Test the PDF code on your computer

This checks the layout without using wxO or COS:

```
pip install reportlab
cd tools
python3 -c "import publish_report_pdf as m; T=m.ReportTable;
open('/tmp/test.pdf','wb').write(m._build_pdf('Test','Total Company',
[T(heading='Regions',summary='East is lowest.',columns=['Region','Actual (M)'],rows=
[['East','20.1']])],'Overall revenue missed budget.'))"
cd ..
open /tmp/test.pdf
```

6.4 Import the tool

```
orchestrate tools import -k python -f tools/publish_report_pdf.py -r tools/requirements.txt --
app-id cos_reports
```

Expected output ends with `Tool 'publish_report_pdf' imported successfully`. If you import it again later, it says `Updating...`.

Check:

```
orchestrate tools list | grep publish_report_pdf
```

6.5 Add the tool to the agent: three edits to the YAML

Open `agents/management_report_generator.agent.yaml` and make these three changes.

Edit 1. Replace the description:

```
description: >
  Generates Budget vs Actual management report packs for the 24Retail Planning
  Analytics database. Pulls revenue and margin data from the Revenue cube, flags
  anomalies, explains variance drivers and publishes a PDF to Cloud Object Storage.
```

Edit 2. In `instructions`, directly after the line that begins `- Before building any breakdown table`, insert this bullet. It must be indented by two spaces like the lines around it, and it is one long line:

```
  - When the user asks for a PDF or report, call publish_report_pdf. Populate tables from data
already retrieved in this conversation. If any data is missing, call
pa_reporting:execute_mdx_and_get_view to fetch it before building the PDF. Pass all tables with
heading, columns, rows and a summary. Every row must have exactly the same number of cells as
columns. All values must be pre-formatted strings in millions with one decimal (for example
```

```
"92.6"), no currency symbol. Include Variance and Variance % columns whenever two versions or periods are compared. The summary field for each table must be 2-4 sentences: state what the table shows, name the member with the largest unfavourable variance and explain its business significance, and name the member with the best performance if applicable. Set commentary to a 2-3 paragraph executive narrative covering overall performance versus Budget and Prior Year, the main variance drivers, any data-quality caveats, and suggested next steps for management. Use only figures already retrieved. Do not include the literal phrase "Draft - for review" in the commentary or any summary string; the tool adds that label automatically. Set subtitle to the entity, version and period. The tool returns download_url, file_name and expires_in_minutes. Present the download_url as a clickable link with the text "Download PDF (valid for 1 hour)". Never re-generate the link unless the user asks again.
```

Edit 3. At the end of the `tools:` list, add:

```
  - publish_report_pdf
```

Re-import:

```
orchestrate agents import -f agents/management_report_generator.agent.yaml
```

Expected output: `Existing Agent 'management_report_generator' found. Updating...` and `updated successfully`.

Part 7: Test the full report flow

Use the preview panel. Start a **new conversation** (reset the chat) so old context does not interfere.

7.1 Run the seven prompts in order, in one conversation

1. Executive summary, 2. Regional, 3. Channel and 4. Product use the prompts from Part 4. Add these:

Prompt 3: channel breakdown

```
Break down 2024 full-year Actual and Budget Gross Revenue by channel: Retail (10), Internet (20), Distribution (30). Millions, Variance vs Budget and Variance %.
```

Prompt 4: product category

```
Break down 2024 full-year Actual and Budget Gross Revenue by product category: Phones (20000), PCs (30000), Tablets (40000). Millions, Variance vs Budget and Variance %.
```

Prompt 5: quarterly trend

```
Show 2024 quarterly Actual and Budget Gross Revenue for Q1, Q2, Q3, Q4 at Total Company level. Millions.
```

Prompt 6: exceptions and drivers

```
Run outlier detection on the 2024 Total Company full-year data across Actual, Budget, Prior Year Actual, Forecast and Target. Then run impact analysis on Actual vs Budget and summarise the top variance drivers.
```

Prompt 7: publish the PDF

```
Generate a PDF report titled "24Retail FY2024 Management Report", subtitle "Total Company - Full Year - All Channels - All Products". Include all tables from this conversation. For each table write a 3-sentence summary explaining what it shows, the biggest unfavourable variance and its business significance. Write a 3-paragraph executive commentary covering overall performance vs Budget and prior year, the main drivers, and suggested management actions. Then publish the PDF and give me the download link.
```

Why this order works: each prompt asks for one topic, so the agent is less likely to drop a column. By prompt 7 all the data is already in the conversation, so the agent calls `publish_report_pdf` once.

7.2 Check the PDF

Click **Download PDF (valid for 1 hour)**. The link is a pre-signed URL on your COS endpoint.

Check	Expected
Title and subtitle	As you typed them
One table per section	Executive summary, regions, channels, product, quarterly
Regions add up	20.1 + 14.0 + 19.1 + 22.9 = 76.1 (Actual)
Variance columns filled	No `N/A`
Summary paragraph above each table	2 to 4 sentences
Executive Commentary section	Headed "Draft - for review", 2 to 3 paragraphs
Footer on every page	"Generated ... AI-assisted - Draft - for review" and a page number
No `**` characters, no black boxes	Bold text is real bold

7.3 Confirm the file is in COS

This step is optional. The bucket belongs to the instructor, so you may not have console access. If you do: IBM Cloud, then the COS instance, then **Buckets**, then the shared bucket. Open the `reports/` folder. You should see the new PDF. Otherwise, opening the download link is enough to confirm the upload.

Part 8: Embed the agent in the PA workspace

You need three things: a **key pair**, the **embed values** from wxO, and the **PA integration screen**.

8.1 Deploy the agent to the live environment

Embedded chat in production uses the **live** variant.

```
orchestrate agents deploy -n management_report_generator
```

If your CLI asks for an environment, it accepts `--env live` (or `-e live`).

8.2 Generate an RSA key pair

Embedded chat is secured with a key pair you create. The **public** key goes to wxO. The **private** key goes to PA.

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

8.3 Register the public key in wxO

1. Open the wxO UI, then **Settings**, then the **Embedded chat** (or **Channels**) security section **(confirm label)**.
2. Paste the public key. Copy it to the clipboard with:

```
cat public_key.pem | pbcopy
```

3. Save and make sure security is **enabled**.

The embedded chat does not start until the keys are configured and validated.

8.4 Get the embed values

Run:

```
orchestrate channels webchat embed --agent-name=management_report_generator --env live
```

If the command asks for your **CRN**, the automatic lookup failed (this happens when the token has expired or the account cannot read the resource list). Get the CRN from IBM Cloud: **Resource list**, open the Orchestrate instance, and copy the **CRN** from the details panel. Paste it at the prompt.

The command prints a `<script>` block. Copy these four values from it:

From the script	Needed in PA as
`orchestrationID`	`orchestrationID`
`hostURL`	`hostURL`
`chatOptions.agentId`	`agentId`
`chatOptions.agentEnvironmentId`	`agentEnvironmentId`

You also need your **purchase ID**. It is a value from your PA subscription, not from wxO. **Instructor to confirm** where students find it. The PA documentation calls it `purchaseID` and describes it as "the unique ID for your purchase or subscription".

8.5 Create the configuration file

Create `embed-agent.json`:

```
{
  "orchestrationID": "<ORCHESTRATION_ID>",
  "hostURL": "https://ca-tor.watson-orchestrate.cloud.ibm.com",
  "agentId": "<AGENT_ID>",
  "agentEnvironmentId": "<AGENT_ENVIRONMENT_ID>",
  "perspectives": [
    "pa-home",
    "pa-administration",
    "dashboard",
    "modeling",
    "modeling-home"
  ],
  "purchaseID": "<PURCHASE_ID>"
}
```

Important format note. The PA documentation shows the values nested inside a `wxo_agent` object, with an `authKey` field. In the lab environment that format was rejected with the error **"Invalid WXO config, orchestrationID is required"**. The format above, with all fields at the top level and **no** `authKey`, is what worked. The private key goes in a separate field (next step).

The `perspectives` list controls where the agent appears. The PA documentation also lists `pa-report` and `pa-plan-contribute`. These two were not tested in the lab.

Check that the file is valid JSON:

```
python3 -c "import json;json.load(open('embed-agent.json'));print('valid')"
```

8.6 Enter the configuration in PA

1. Sign in to Planning Analytics.
2. In the top bar, open the **Administration** menu (the dropdown next to the gear icon) and choose **Administration**. **(confirm label)**

3. Open **Integrations**.
4. In the list, select **IBM watsonx Orchestrate**. Its status should be **Active**.
5. Select **Custom instance**.
6. Under **Configuration value**, clear any text, then paste the JSON:

```
cat embed-agent.json | pbcopy
```

7. Under **Embedded chat authorization key**, paste the **raw** private key, including the `-----BEGIN PRIVATE KEY-----` and `-----END PRIVATE KEY-----` lines and real line breaks. Do **not** convert line breaks to `\n`. Copy it with:

```
cat private_key.pem | pbcopy
```

8. Click **Test**. Fix any error message, then click **Apply**.

After pasting the private key, copy something harmless to overwrite your clipboard.

8.7 Test inside PA

1. Open a perspective you included, for example **Home**.
2. Open the chat (assistant) panel.
3. Confirm the greeting comes from your agent **(confirm how to tell)**.
4. Run Prompt 1 from Part 4.
5. Run Prompt 7 from Part 7 and open the link.

Things to know about the embedded agent

- The agent connects to PA through **one shared account** (the `pa_mcp_credentials` Basic connection). Every user of the embedded agent therefore sees the data that account can see. PA's per-user data security does not apply to the agent. Decide if that is acceptable before using real data.
- Pre-signed links are valid for one hour. After that, ask the agent to produce a new report.
- PA's documentation says OAuth-type connections are not supported in embedded web chat. Basic and key_value connections are what this lab uses.

Part 9: Troubleshooting

Symptom	Cause	Fix
`The token found for environment ... is missing or expired`	Login expired	Repeat Part 1.3 (`read -s`, `activate`, `unset`)
`401 Unauthorized` when importing the toolkit	Connection is not Basic, or password is wrong	`orchestrate connections list`. Recreate as `--kind basic` with username `apiKey`
`orchestrate chat ask` stops after 90 to 180 seconds	First calls are slow	Test in the wxO preview panel instead
Agent says it cannot use Data Explorer	The Revenue cube is not pre-analysed	The agent uses `execute_mdx_and_get_view` instead. Nothing to fix
`AttributeError: 'dict' object has no attribute 'rows'`	The tool treated tables as objects	Keep the dict-to-`ReportTable` loop in `publish_report_pdf`
Variance columns show `N/A` or Budget row missing	Conflicting instructions, or the agent dropped data	Use the variance rule in Part 3.1. Ask for the Budget version explicitly
Regions do not add up to Total Company	The agent looked up a region by alias	Use member IDs 100, 200, 300, 400 in the prompt
Black box in the PDF text	A character that Helvetica cannot draw	Keep `_safe()` in the tool
`**text**` shown in the PDF	Markdown not converted	Keep `_inline()` in the tool
"Draft - for review" appears twice	Agent put the label in the commentary	Keep the instruction not to add it, and the filter in the tool
`AccessDenied` or signature errors from COS	Wrong keys, endpoint or region	Re-check the five values with your instructor and re-run Part 5.2
`NoSuchBucket`	Wrong `COS_BUCKET` or endpoint	Re-run Part 5.2 with the correct values
PA shows "Invalid WXO config, orchestrationID is required"	Wrong JSON shape, or leftover text in the box	Use the flat format in Part 8.5. Clear the box with Cmd+A then paste
Embedded chat does not load	Public key not registered or security not enabled in wxO, or the private key does not match	Redo Part 8.2 and 8.3 with a fresh pair, and update PA
`orchestrate channels webchat embed` asks for a CRN	Automatic lookup returned 401	Re-activate the environment, or paste the CRN from IBM Cloud

Part 10: Clean-up

Delete local key files and temporary files:

```
rm -f private_key.pem public_key.pem embed-agent.json
```

The shared COS bucket and credential belong to the instructor. Do not delete them.

In wxO:

1. Remove the test agent and tools if the account is shared:

```
orchestrate agents remove -n management_report_generator -k native
orchestrate tools remove -n publish_report_pdf
orchestrate toolkits remove -n pa_reporting
orchestrate connections remove -a pa_mcp_credentials
orchestrate connections remove -a cos_reports
```

2. Delete the wxO API key you created in Part 1.2 (Settings, API details).

In PA:

1. Administration, Integrations, IBM watsonx Orchestrate: switch back to **Default instance** and apply.
2. Revoke the PA API key if it was created for this lab.

Appendix A: File layout at the end of the lab

```
lab-1203/
├── .venv/
├── agents/
│   └── management_report_generator.agent.yaml
├── tools/
│   ├── publish_report_pdf.py
│   └── requirements.txt
└── embed-agent.json          (delete in Part 10)
```

Appendix B: Resources created

Where	Name	Type
wxO	`lab-1203`	ADK environment (local name)
wxO	`pa_mcp_credentials`	Basic connection (team scope, draft and live)
wxO	`pa_reporting`	Remote MCP toolkit, 8 tools
wxO	`cos_reports`	key_value connection
wxO	`publish_report_pdf`	Python tool
wxO	`management_report_generator`	Native agent
IBM Cloud	shared COS bucket	Created by the instructor before the class
PA	Integrations, IBM watsonx Orchestrate	Custom instance configuration

Appendix C: Command quick reference

Task	Command
Activate environment	`orchestrate env activate lab-1203 --api-key "$WXO_KEY"`
List connections	`orchestrate connections list`
Add a connection	`orchestrate connections add -a <APP_ID>`
Configure a connection	`orchestrate connections configure -a <APP_ID> --env draft --type team --kind <basic or key_value>`
Set Basic credentials	`orchestrate connections set-credentials -a <APP_ID> --env draft -u <USER> -p "$PASSWORD"`
Set key_value credentials	`orchestrate connections set-credentials -a <APP_ID> --env draft -e "KEY=value"`
Add a remote MCP toolkit	`orchestrate toolkits add --kind mcp --name <NAME> --description "<TEXT>" --url <URL> --transport streamable_http --tools "<LIST>" --app-id <APP_ID>`
Import a Python tool	`orchestrate tools import -k python -f <FILE> -r <REQUIREMENTS> --app-id <APP_ID>`
Import or update an agent	`orchestrate agents import -f <YAML>`
Deploy an agent	`orchestrate agents deploy -n <NAME>`
Get embed script	`orchestrate channels webchat embed --agent-name=<NAME> --env live`